SikhyaSetu

The offline misconception-to-mastery tutor for Odia-medium learners

Hackathon proposal — polished build-ready version
Format: software-only, local-first PWA + cloud agent backend
Primary learner: Sunita, Class 9, Odia-medium, shared phone, unreliable connectivity

> **One-line pitch:** SikhyaSetu turns every wrong answer into a diagnosis, every diagnosis into a tiny offline lesson, and every offline session into the teacher's next best action — in Odia and within the learner's data budget.

---

1. Executive verdict

Is the current idea strong?

Yes — the core idea is genuinely differentiated and technically credible. The strongest parts of the attached concept are:

1. **Compile online, serve offline:** cloud agents prepare a compact personalised lesson pack; the phone runs it without a live model or signal.
2. **Misconception diagnosis rather than scorekeeping:** the product asks *why* a learner is wrong and selects a counter-example or prerequisite lesson.
3. **Deferred intelligence:** voice teach-back can be recorded offline and graded when connectivity returns.
4. **Data- and time-aware planning:** the next pack is sized around the learner's available megabytes and study window.
5. **A real agent loop:** events → diagnosis → mastery update → plan → pack → next session.

What would weaken it in a competitive field?

The original document lists seven strong ideas, but gives them similar prominence. A judge may remember none of them, or may conclude that the team is promising too much:

- peer-to-peer sharing, peer voice notes, try-it verification, live doubt solving, nudges, shared-phone profiles, voice, teacher analytics, and a full cloud agent loop are too much for one hackathon MVP;
- the phrase "AI tutor" is common, so the differentiator must be the **offline misconception loop**, not the existence of a chatbot;
- several integrations are described before the minimum end-to-end proof is locked;
- the product needs explicit acceptance criteria and a fallback demo path.

Recommended strategic decision

Keep the architecture, narrow the story. Make one loop the hero:

> **Probe → explain → teach back → diagnose misconception → compile the next offline pack.**

Make three distinctive MVPs visible:

1. **Misconception Fingerprint:** a structured learner state that records the likely wrong mental model, not merely a score.
2. **Learning-per-MB Compiler:** an agent that chooses the most valuable next content under a data and time budget.
3. **Teacher Action Brief:** a class-level summary that turns aggregated misconceptions into one suggested five-minute classroom intervention.

Shared-phone profiles remain an important enabling feature. Peer sharing, try-it tasks, WhatsApp nudges, and live free-form doubt answering become roadmap items unless already working.

---

2. The problem

Sunita is a Class 9 learner in an Odia-medium setting. She may have access to a shared Android phone for roughly one hour in the evening, but not to a stable internet connection, unlimited data, or an always-available tutor.

Existing learning apps often assume:

- continuous connectivity;
- video-heavy consumption;
- individual device ownership;
- comfortable English reading and typing;
- a teacher who can interpret a score and decide what to do next.

The result is a broken learning loop:

> **She answers incorrectly, but nobody identifies the mental model behind the error or gives her a targeted next step.**

Product gap in one sentence

SikhyaSetu gives an Odia-medium learner a personalised tutor that works offline, identifies the misconception behind an answer, and uses the next connection to prepare only the most valuable content for her next session.

Primary and secondary users

User	Need	SikhyaSetu response
Learner	Understand difficult concepts in Odia despite weak connectivity	Offline lesson packs, audio/text, probes, teach-back, targeted intervention
Teacher	Know which misconceptions are common and what to do tomorrow	Teacher Action Brief and misconception heatmap
Programme/cluster coordinator	Deliver learning support under data and device constraints	Small signed packs, syncable events, privacy-preserving aggregate insights

---

3. Product thesis: one loop, three innovations

3.1 The hero loop: Misconception-to-Mastery

1. **Probe:** the learner answers a carefully designed question.
2. **Fingerprint:** the selected distractor or teach-back is mapped to a likely misconception.
3. **Intervene:** the offline tutor serves a counter-example, analogy, prerequisite, or worked example.
4. **Teach back:** the learner explains the idea in her own words; audio can be queued offline.
5. **Sync and grade:** when a connection returns, the cloud grader evaluates the teach-back against a rubric.
6. **Re-plan:** the planner chooses the next concept and compiles a new compact pack.

This is not “chat with an LLM.” The agents create a closed learning system that changes what the learner sees next.

3.2 MVP 1 — Misconception Fingerprint

Each learner has a small, explainable state per concept:

- current mastery estimate;
- likely misconception label;
- evidence: probe choice, confidence, time, teach-back rubric;
- recommended intervention;
- confidence in the diagnosis;
- next review date.

Example:

> **Concept:** force and motion
> **Observed error:** treats force as something an object “contains”
> **Evidence:** selected distractor B + teach-back phrase pattern
> **Intervention:** bicycle-braking counter-example + one prerequisite micro-lesson
> **Next check:** same concept with a new context after spaced revision

This gives judges a concrete answer to “How is this more than a quiz app?”

3.3 MVP 2 — Learning-per-MB Compiler

The Planner/Compiler agent does not simply generate a long lesson. It selects the highest-value learning items under constraints:

- estimated available data;
- learner’s typical session duration;
- unresolved misconceptions;
- prerequisites;
- spaced-revision needs;
- audio/text size;
- content confidence and review status.

A pack can contain **text-first essentials**, optional compressed audio, and a small number of high-information probes. The system can show a judge:

> "We had 2.4 MB and 35 minutes. We prioritised the misconception repair and one prerequisite, deferred optional audio, and scheduled revision for tomorrow."

This turns low bandwidth from a limitation into an explicit agent decision.

3.4 MVP 3 — Teacher Action Brief

The teacher view should not be a generic dashboard. It should answer one practical question:

> **What can the teacher do in the next five minutes to address the class's biggest misconception?**

The brief contains:

- top three misconception clusters;
- number of learners affected, shown as a count or bucket rather than personally identifying data;
- one suggested classroom prompt or demonstration;
- learners needing follow-up, if consent and identification are enabled;
- what the planner will send in the next pack cycle.

Example:

> "12 learners are treating current as something that gets used up. Use the two-bulb comparison prompt, then ask learners to explain where the brightness changes. SikhyaSetu will send a counter-example pack to the affected profiles."

---

4. What is in the hackathon MVP

Must work end to end

- One subject: **Class 9 Science**.
- One or two tightly bounded chapters.
- 10–15 concepts and 8–10 misconception patterns.
- Two learner profiles on one phone.
- A pre-generated Odia lesson pack stored in the PWA.
- Full offline lesson flow with network switched off.
- At least three probe questions mapped to known misconceptions.
- Offline teach-back recording queued for later grading.
- Cloud sync that updates the learner state and compiles the next pack.
- Visible agent trace: event received → diagnosis → plan → budget → compile.
- Minimal Teacher Action Brief using the same demo events.

Should work if time permits

- Native Odia TTS or reviewed audio for the hero flow.
- A live online doubt box restricted to retrieved textbook content.
- Pack size preview and "why this content was selected" explanation.
- Manual "Sync now" button for browser reliability.

Explicitly defer

- Peer-to-peer pack sharing.
- Peer voice-note matching.
- WhatsApp/SMS nudges.
- Automated household experiment verification.
- Broad multi-subject support.
- Claims of a fully offline generative tutor.

Reason: these are valuable roadmap ideas, but they dilute the proof of the core loop and add integration risk.

---

5. User experience

Learner flow — offline

1. **Select profile:** large icon/avatar; no photo required.
2. **Today's plan:** "2 concepts, 25 minutes, 0.8 MB already downloaded."
3. **Learn:** short Odia explanation with a local example; English technical term shown where useful.
4. **Probe:** one question with distractors linked to misconceptions.
5. **Diagnosis:** "Your answer suggests this idea is mixed up…" in respectful language.
6. **Repair:** counter-example, analogy, or prerequisite mini-lesson.
7. **Teach back:** record a short explanation in Odia; show "saved for next sync."
8. **Progress:** show concept status locally, without pretending that deferred voice grading is complete.

Online reconnect flow

1. Sync events using unique IDs and retry-safe batches.
2. Transcribe and grade queued teach-backs.
3. Update the Misconception Fingerprint.
4. Generate the Teacher Action Brief.
5. Run the planner under a data/time budget.
6. Compile and deliver the next signed pack.

Language design

- Native Odia generation or human-reviewed demo content; do not present raw machine translation as finished UX.
- Short sentences, large type, audio-first controls, minimal typing.
- Local examples such as paddy fields, hand pumps, bicycles, rainfall, and household electricity.
- Code-mixing only where it helps connect classroom terminology to Odia explanation.
- Respectful diagnosis: "Let's check this idea again," not "You are wrong."

---

6. Architecture and agency

SikhyaSetu has **two layers** with a clear division of responsibility:

Layer 1 — The phone: reliable offline execution

The phone contains the learner's profiles, downloaded lesson packs, local progress, and queued events. A small deterministic tutor runs the pack without an internet connection:

Select profile → teach → probe → diagnose → intervene → record → log

The phone does not need to call an LLM during the lesson. This makes the core experience fast, predictable, and usable in airplane mode.

Layer 2 — The cloud: adaptive planning when connectivity returns

The cloud receives the phone's events and performs the work that needs deeper reasoning:

Sync → grade teach-back → update learner state → plan → optimise budget → compile pack → deliver

The cloud also generates the class-level **Teacher Action Brief** from aggregated misconception evidence.

End-to-end data flow

Step	What happens	Where
1. Prepare	The planner chooses concepts, prerequisites, and revision items	Cloud
2. Compile	The compiler creates a small, structured Odia lesson pack within the MB/time budget	Cloud
3. Download	The signed pack is downloaded in a small, resumable transfer	Phone ↔ cloud
4. Learn offline	The deterministic tutor delivers the lesson, probe, diagnosis, and intervention	Phone
5. Collect evidence	Answers, time, progress, and teach-back audio are stored locally	Phone
6. Reconnect	Events and queued audio are synchronised safely using unique IDs	Phone ↔ cloud

Step	What happens	Where
7. Adapt	The grader updates the Misconception Fingerprint and the planner creates the next pack	Cloud
8. Support the teacher	Aggregated evidence becomes a five-minute classroom recommendation	Cloud

Simple architecture view

PHONE — works offline	CLOUD — used when online
Local-first React/TypeScript PWA	FastAPI sync and pack API
IndexedDB: profiles, packs, events, audio queue	Grader: transcription and teach-back rubric
Service worker: cached app shell	Misconception state and mastery update
Deterministic tutor state machine	Planner and Learning-per-MB Compiler
MediaRecorder for offline teach-back	Pack store and Teacher Action Brief

Only small, resumable sync batches and compressed lesson packs move between the two layers. The learner's session does not stop when the network disappears.

Where the agency lives

Component	Role	Why it is agentic or deterministic
Cloud Planner	Selects concepts, prerequisites, revision, and follow-up	Open-ended prioritisation based on evidence and constraints
Budget Optimiser	Chooses content mix under MB/time limits	Explicit trade-off and tool decision
Pack Compiler	Generates structured Odia content and interventions	Produces a personalised executable pack
Grader	Evaluates delayed teach-back against a rubric	Converts unstructured evidence into a diagnosis
Phone tutor	Executes the pack offline	Deterministic, reliable, auditable, and lightweight
Sync layer	Reconciles events and retries safely	Operational reliability, not free-form generation

Why not put an LLM on the phone?

A live on-device model would make the demo fragile, increase storage and performance requirements, and make Odia quality difficult to guarantee. SikhyaSetu instead compiles intelligence when a connection exists and executes a tested pack offline.

---

7. Pack and event design

Section 7 defines the two small data objects that make the local-first design possible:

1. **The lesson pack:** what the phone needs in order to teach offline.
2. **The event log:** what the phone records so the cloud can adapt the next session.

7.1 What is inside a lesson pack?

A lesson pack is a compact, signed, learner-specific bundle. It is not a full textbook and it does not contain a live chatbot. It contains only the next best learning experience selected by the planner.

Pack component	Purpose
Pack ID and version	Identifies the learner's current pack and prevents old content from overwriting new content
Language and reading level	Keeps explanations appropriate for the learner and the selected language
Learning objectives	Defines the concepts the learner should work on in this session
Short explanations	Provides text and, where available, reviewed Odia audio
Local examples	Connects the concept to familiar situations such as bicycles, rainfall, farming, or household electricity

Pack component	Purpose
Probe questions	Tests understanding with answer choices linked to specific misconceptions
Intervention cards	Contains the counter-example, analogy, worked example, or prerequisite lesson to show after an error
Teach-back prompt	Asks the learner to explain the concept in her own words
Budget metadata	Records the pack's expected size in KB and expected completion time
Integrity information	Allows the phone to verify that the pack was not corrupted or tampered with

7.2 How the pack is used offline

Before connectivity is lost, the cloud delivers the pack to the phone. During the lesson, the phone follows the pack in a predictable order:

Load pack → show explanation → ask probe → map response to misconception → show intervention → request teach-back → save progress

The phone can complete this entire flow without calling an LLM. This is why the demo continues to work in airplane mode.

7.3 What is an event?

An event is a small record of something important that happened during the learner's session. Events are written locally first and uploaded later.

Event	Meaning
`session_started`	The learner began a pack
`probe_answered`	The learner selected an answer and, when available, confidence or time was recorded
`intervention_viewed`	The learner received a targeted explanation or counter-example
`teachback_recorded`	A voice or text explanation was saved for later grading
`pack_completed`	The learner reached the end of the planned session
`sync_succeeded`	A batch was accepted by the cloud
`sync_failed`	A batch could not be uploaded and must be retried

7.4 How offline sync works

1. The phone writes each event immediately to IndexedDB.
2. Each event receives a unique ID, so a retry cannot create a duplicate.
3. When connectivity returns, the phone uploads events in small batches.
4. The cloud acknowledges accepted events and processes the new evidence.
5. The grader updates the Misconception Fingerprint.
6. The planner compiles the next pack.
7. The phone downloads the new pack and keeps the previous pack until the new one is verified.

This means a dropped connection does not erase progress and does not force the learner to restart.

7.5 Example: one learner event becoming a new decision

Evidence	Cloud interpretation	Next action
Sunita selects distractor B on a force question	Possible "force-as-substance" misconception	Send a bicycle-braking counter-example
She records a teach-back with low rubric confidence	Diagnosis needs confirmation	Schedule a second probe in a new context
She has 35 minutes and 1.2 MB available	Optional audio may not fit the budget	Send text-first essentials and defer optional audio

The important point is that events are not collected only for analytics. They directly change the next lesson.

7.6 Privacy and integrity safeguards

- No learner photographs are required.
- Use pseudonymous learner IDs in the prototype.
- Store only the minimum data needed for learning adaptation.

- Make audio retention configurable; delete audio after grading where possible.
- Separate class-level aggregates from identifiable learner views.
- Verify pack integrity before activation on the phone.
- State clearly that the prototype is not a clinical, exam, or high-stakes assessment system.

---

8. Failure handling and honest capability boundaries

Failure	User-visible behaviour
No connection	Current pack continues; events and audio queue locally
STT unavailable	Keep audio; offer a simple tap-based response fallback
Low-confidence voice grading	Mark “needs review”; do not invent a diagnosis
Cloud LLM unavailable	Retry, then use reviewed template/cached pack; show sync pending
Pack download interrupted	Resume by chunk or keep the previous pack
Storage constrained	Drop old audio first; preserve learner state and event log
Browser background sync unavailable	Provide a visible “Sync now” action
Missing content for a doubt	Log the gap and show the closest reviewed passage

Claims we can make in the demo

- “The lesson flow and misconception probe run offline.”
- “Teach-back is recorded offline and graded after reconnect.”
- “The cloud planner changes the next pack using the learner’s evidence.”
- “The pack compiler makes a visible data/time trade-off.”

Claims we should not make yet

- “The tutor understands every Odia voice answer offline.”
- “The system improves exam scores” without a real evaluation.
- “The app works on every low-end phone” before device testing.
- “The agent is fully autonomous” without showing tools, state, constraints, and fallback paths.

---

9. Recommended technical stack

Layer	Recommendation	MVP rule
PWA	React + TypeScript	Prioritise offline shell and deterministic flow
Local storage	IndexedDB via Dexie	Store packs, events, profiles, and audio queue
Offline execution	Small state machine	No network dependency during the hero demo
Backend	FastAPI	Keep API surface small and observable
Agent orchestration	LangGraph or equivalent explicit graph	Show state and tool calls, not hidden prompts only
Database	SQLite for demo; PostgreSQL if already available	Avoid infrastructure work that does not improve the demo
Retrieval	Small curated textbook passage index	Keep source passages visible and cited
Speech	Reviewed prerecorded Odia audio first; STT as deferred feature	Do not let a provider outage break the story
Hosting	Any reliable free-tier API/static host	Add manual sync control

Important implementation choice: prepare the demo’s Odia content in advance and allow the compiler to generate at least one new pack from structured templates. This protects the team from LLM latency or language-quality surprises while still demonstrating genuine adaptation.

10. The five-minute winning demo

Before the demo

- Load Sunita's first pack and a second learner profile.
- Prepare one known misconception and one reviewed Odia intervention.
- Keep a network toggle or browser devtools offline control ready.
- Seed the cloud with a small concept graph and rubric.

Live sequence

0:00–0:35 — Story and promise
Show Sunita's constraint: one shared phone, weak connectivity, Odia-medium learning. State the one-line pitch.

0:35–1:45 — Offline learning
Select Sunita. Turn on airplane mode. Complete a lesson. Deliberately choose a wrong distractor. Show the misconception fingerprint and targeted intervention.

1:45–2:20 — Teach-back
Record a short Odia explanation. Show that it is saved locally and queued, not falsely graded in the absence of a connection.

2:20–2:40 — Shared-phone mode
Switch to a second profile. Show that progress and packs are isolated.

2:40–4:15 — Agent reconnect
Restore connectivity. Show the event sync, deferred grading, mastery update, planner decision, budget decision, and newly compiled pack. Keep the trace human-readable:

> "Because Sunita selected the force-as-substance distractor and has 35 minutes with 1.2 MB available, we prioritised counter-example 7, deferred optional audio, and scheduled a review tomorrow."

4:15–5:00 — Teacher Action Brief
Show the class misconception cluster and the suggested five-minute intervention. End by returning to the problem: the system does not only teach; it closes the loop between learner evidence, next content, and teacher action.

Demo backup path

If the live compiler or STT fails, switch to a seeded response while clearly labelling it as a fallback. The offline flow, trace, and pack transition must remain demonstrable without external provider availability.

---

11. Build plan

Phase 1 — Foundation

- Freeze one chapter and define the concept graph.
- Write 8–10 misconception patterns and associated distractors.
- Define pack and event schemas.
- Implement the PWA shell, two profiles, and IndexedDB.

Phase 2 — Hero loop

- Implement offline teach/probe/diagnose/intervene flow.
- Add local progress and event logging.
- Add teach-back recording queue.
- Create the first reviewed Odia pack.

Phase 3 — Cloud intelligence

- Implement sync endpoint and idempotent event ingestion.
- Implement grader rubric and misconception-state update.
- Implement planner and budget optimiser with visible trace.
- Compile the next pack from the updated state.

Phase 4 — Judge-facing polish

- Build Teacher Action Brief.
- Add pack size and decision explanation.

- Test with network disabled, interrupted sync, empty storage, and provider failure.
- Rehearse the five-minute flow on the actual demo phone.

Cut order if time runs short

1. Cut live free-form doubt answering.
2. Cut TTS generation; use reviewed audio or text.
3. Cut teacher learner-level drill-down; keep class-level brief.
4. Cut shared-phone voice profile switching; keep icon switching.
5. **Never cut:** offline flow, misconception diagnosis, event sync, visible agent trace.

---

12. Evaluation plan and measurable prototype targets

These are engineering targets, not claims of educational impact:

- Offline core flow starts and completes with network disabled.
- At least 90% of seeded test events survive an interrupted sync without duplication or loss.
- Every demo misconception has a deterministic mapping to an intervention.
- New pack generation displays the selected concept, reason, estimated size, and time budget.
- Teacher Action Brief is generated from actual demo events, not a static screenshot.
- All demo Odia text/audio is reviewed by a native speaker or clearly labelled as prototype output.
- Pack size remains within the chosen budget for the hero scenario.
- The app can recover from a failed provider call using a reviewed fallback.

A future pilot could measure misconception resolution across repeated probes, teach-back rubric improvement, completion rate under data limits, and teacher usefulness. Do not present those as proven during the hackathon.

---

13. Judge questions and strong answers

“Why is this agentic and not just a chatbot?”

Because the system observes events, updates a structured learner state, chooses the next concepts, optimises content under constraints, compiles an executable pack, and schedules follow-up. The offline tutor is the reliable executor; the cloud agents make the decisions that change the next session.

“Why not use an LLM offline?”

The product needs reliability on low-end phones and honest Odia quality. We compile the model’s work online into a tested pack, then run that pack offline. This is more practical and auditable than pretending a live generative tutor is always available.

“What is your most defensible innovation?”

The **Misconception-to-Mastery loop under a data budget**. Offline support alone is not new; diagnosis alone is not new. The combination closes the loop despite intermittent connectivity.

“What happens when the learner has no internet?”

She can complete the downloaded lesson, answer probes, receive deterministic interventions, record a teach-back, and continue. The intelligence that requires the cloud is deferred rather than silently fabricated.

“How do you handle incorrect AI output?”

Content is generated into a structured pack, constrained by a curated concept/misconception library, reviewed for the demo, and served with fallback templates. The phone does not freely generate high-stakes explanations offline.

“Can this scale beyond Odia?”

The pack schema, misconception state, and deterministic tutor are language-agnostic. The next language requires reviewed content, speech support, and local examples; it is a roadmap, not an unsupported current claim.

“How do you protect children’s data?”

Use pseudonymous learner IDs, collect minimal events, avoid photographs, separate aggregates from identifiable views, and make audio retention explicit. A real deployment would require institutional consent and a fuller child-data governance process.

---

14. Risks and mitigations

Risk	Mitigation
Odia generation quality	Review hero content with a native speaker; use structured templates
STT/TTS provider outage	Treat speech grading as deferred; keep tap/text fallback and seeded audio
Browser storage/background-sync differences	Use a visible sync button and test on the actual phone
Over-scoping	Protect the one loop; defer integrations and secondary features
Weak misconception library	Start with fewer, high-quality patterns rather than many shallow ones
Misleading learning claims	Report prototype behaviour and engineering metrics only
Textbook licensing	Use permitted or team-authored excerpts and cite sources
Child privacy	Pseudonymous IDs, minimum data, configurable deletion, consent plan

---

15. Final positioning

The sentence to repeat throughout the pitch

> **SikhyaSetu is not an online chatbot that fails when the signal disappears. It is an agentic learning loop that uses the cloud to prepare the next best lesson and the phone to deliver it offline.**

The three features judges should remember

1. **Misconception Fingerprint:** it identifies the wrong idea, not just the wrong answer.
2. **Learning-per-MB Compiler:** it spends scarce data on the highest-value next step.
3. **Teacher Action Brief:** it converts learner evidence into tomorrow's five-minute classroom action.

Final recommendation

Build the revised version. The original concept is strong enough to compete, but the winning version is **smaller, sharper, more honest, and easier to prove live**. Make the offline misconception loop unavoidable in the demo; treat everything else as support for that loop or as a clearly labelled roadmap.

Appendix A — Suggested project name and tagline options

- **SikhyaSetu — The next lesson, even when the signal is gone.**
- **SikhyaSetu — Learn offline. Diagnose deeply. Improve deliberately.**
- **SikhyaSetu — From wrong answer to right next step.**

Appendix B — What was reviewed

This document was prepared from the supplied file **"SikhyaSetu (Software-Only Edition)"**. No separate original problem-statement attachment was available in the received files. The next revision should cross-check all user/problem statistics, judging criteria, and required technology constraints directly against the official problem statement before submission.